

UNITÉ 3 · SEMAINES 3–4

Fichiers et systèmes de fichiers

📅 Semaines 3–4 · dates à confirmer </> Laboratoire TD2 (corrigé + défi IA) ~ 4 animations

MANUEL M3 · THÉORIE DE L'UNITÉ

Fichiers et systèmes de fichiers

Lecture de base de l'unité 3 (≈ 1 h 30). Chaque section se termine par un lien **Pratiquez** →.

Les sorties sont réelles: elles ont été obtenues avec les programmes du TD2 sur la machine du cours (Ubuntu 26.04, Linux 7.0, ext4 sur NVMe).

SOMMAIRE

- | | | | |
|----|---|----|---|
| §1 | Deux niveaux: le disque et le noyau | §5 | Fichiers spéciaux: FIFO, périphériques, <code>/proc</code> |
| §2 | L'inode: métadonnées et blocs | §6 | Parcourir les répertoires |
| §3 | Les noms: répertoires, liens et chemins | §7 | Au-delà: journalisation, copie sur écriture et <code>fsync</code> |
| §4 | Fichiers ouverts: les trois tables | | |

§1 Deux niveaux: le disque et le noyau

Un fichier existe à deux endroits. **Sur le disque**, le système de fichiers organise un immense tableau de blocs (4 Kio sur la machine du cours): un *superbloc* avec la description générale, des bitmaps qui indiquent quels blocs et inodes sont libres, les *tables d'inodes* et les *blocs de données*. ext4 répète cette structure en *groupes de blocs* le long de la partition, avec des copies du superbloc. Le nombre d'inodes est fixé au formatage:

Sortie réelle

```
$ df -iT . | cut -c1-80; stat -f -c 'block size: %S bytes' .
Filesystem      Type  Inodes   IUsed   IFree IUse% Mounted on
/dev/nvme0n1p5 ext4 25600000 2372366 23227634  10% /
block size: 4096 bytes
```

Dans le noyau, tant qu'un processus utilise le fichier, il y a trois tables: la table des descripteurs de chaque processus, la table système des fichiers ouverts et la table des inodes en mémoire (§4). Par-dessus tout, la *cache de pages* garde en RAM les blocs lus récemment: c'est pourquoi la deuxième lecture d'un fichier ne touche presque jamais le disque.

Voir dans le diaporama P3 →

§2 L'inode: métadonnées et blocs

Unix sépare le **contenu** d'un fichier de ses **métadonnées**. Les métadonnées vivent dans un **inode**, identifié par un numéro: type et permissions (`st_mode`), nombre de liens physiques (`st_nlink`), propriétaire et groupe, taille, périphérique, horodatages et liste des blocs. L'inode ne contient **pas** le nom. `stat`, `fstat` (sur un descripteur) et `lstat` (sans suivre les liens) le lisent:

Sortie réelle

```
$ stat notas.txt
  File: notas.txt
  size: 11          Blocks: 8          IO Block: 4096   regular file
Device: 259,5      Inode: 12190686    Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/student)   Gid: ( 1000/student)
Access: 2026-09-30 13:12:49.872600645 -0500
Modify: 2026-09-30 13:12:49.872600645 -0500
Change: 2026-09-30 13:12:49.879601063 -0500
 Birth: 2026-09-30 13:12:49.872600645 -0500
```

Les quatre horodatages: **Access** (dernière lecture; Linux le met à jour paresseusement avec `relatime`), **Modify** (dernière modification du contenu), **Change** (dernière modification de l'inode : permissions, liens, propriétaire... ou contenu) et **Birth** (création, disponible sur ext4 et lue avec `statx`). C'est une erreur fréquente — les diapositives d'origine la faisaient — de croire que `ctime` est la création.

On teste le type avec les macros `S_ISREG`, `S_ISDIR`, `S_ISLNK`, `S_ISCHR`, `S_ISBLK`, `S_ISFIFO` et `S_ISSOCK` sur `st_mode`. Les erreurs typiques de `stat` sont `ENOENT` (n'existe pas), `EACCES` (droit de traversée `x` manquant sur un répertoire du chemin), `ENOTDIR` et `ELOOP`.

Où sont les blocs? ext2 et ext3 stockaient 15 pointeurs dans l'inode: 12 directs, un indirect simple (un bloc rempli de pointeurs), un double et un triple. Avec des blocs de 4 Kio et des pointeurs de 4 octets, 1024 tiennent par bloc et le schéma atteint environ 4 Tio, mais une position lointaine oblige à lire jusqu'à 3 blocs d'index. ext4 utilise des **extents**: des zones contiguës décrites par (début logique, longueur ≤ 32 768 blocs, début physique). Quatre tiennent dans l'inode; s'il en faut plus, on construit un arbre. Le fichier de 10 Mo écrit par le TD2 n'en occupe que deux:

Sortie réelle

```
$ head -c 10000000 /dev/urandom > big.bin; sync big.bin; filefrag -v big.bin
Filesystem type is: ef53
File size of big.bin is 10000000 (2442 blocks of 4096 bytes)
ext:      logical_offset:      physical_offset: length:   expected: flags:
  0:         0..      2047:   15654912..   15656959:    2048:
  1:      2048..      2441:   15673843..   15674236:    394:   15656960: last,eof
big.bin: 2 extents found
```

Pratiquez → A3.4 De l'inode aux blocs · TD2 · 1

§3 Les noms: répertoires, liens et chemins

Un **répertoire** est un fichier dont les données sont une liste de paires *nom* → *numéro d'inode*, plus **.** (lui-même) et **..** (son parent). Pour ouvrir `/home/student/notas.txt`, le noyau part de l'inode racine (numéro 2 sur ext4), lit le répertoire, cherche `home`, continue avec son inode, et ainsi de suite composant par composant (*path_resolution(7)*). Chaque composant exige le droit de traversée (**x**) sur le répertoire. Linux garde les résolutions récentes dans le *cache des dentries*.

Un **lien physique** (`link`, `ln`) est une autre entrée qui pointe vers le **même** inode: les deux noms sont aussi « originaux » l'un que l'autre et `st_nlink` les compte. Ils sont interdits vers les répertoires (ils casseraient l'arbre) et entre systèmes de fichiers (les numéros d'inode sont locaux). Un **lien symbolique** (`symlink`, `ln -s`) est un nouvel inode dont le contenu est un *chemin*; la résolution le suit (au plus 40 fois: au-delà, `ELOOP`) et il peut rester pendant si la cible disparaît.

Sortie réelle

```
$ ln notas.txt copia.txt; ln -s notas.txt atajo; ls -li notas.txt copia.txt atajo
12190687 lrwxrwxrwx 1 student student 9 Sep 30 13:12 atajo → notas.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 copia.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 notas.txt
```

`rm` appelle `unlink`, qui supprime une **entrée** et retire 1 à `st_nlink`. Le noyau ne libère l'inode et ses blocs que lorsque `st_nlink = 0` **et** qu'aucun processus ne l'a ouvert. En attendant, le fichier reste lisible par son descripteur:

Sortie réelle

```
$ exec 3< notas.txt; rm notas.txt copia.txt; readlink /proc/$$/fd/3; cat <&3; ls -l
atajo; cat atajo
~/fso/tds/tests/tmp/td2/notas.txt (deleted)
hola inodo
lrwxrwxrwx 1 student student 9 Sep 30 13:12 atajo → notas.txt
cat: atajo: No such file or directory
[exit status 1]
```



Vérifier puis utiliser: la course TOCTOU

if (access(path, W_OK) == 0) fd = open(path, O_WRONLY); laisse une fenêtre: entre les deux appels, un autre processus peut remplacer `path` par un lien vers un autre fichier. Ouvrez directement et gérez l'erreur (`EACCES` , `ENOENT`), ou utilisez `O_NOFOLLOW` ou `openat2()` avec `RESOLVE_NO_SYMLINKS` . De plus, `access` vérifie avec l'UID *réel*, pas l'effectif.

Pratiquez → A3.1 Inodes et répertoires

§4 Fichiers ouverts: les trois tables

`open(path, flags, mode)` résout le chemin, vérifie les permissions et renvoie le **plus petit descripteur libre**. Les drapeaux: accès (`O_RDONLY` , `O_WRONLY` , `O_RDWR`), création (`O_CREAT` , `O_EXCL` , `O_TRUNC`); les permissions demandées passent par le `umask`) et état (`O_APPEND` , `O_NONBLOCK` , `O_CLOEXEC`). La glibc implémente `open` avec `openat(AT_FDCWD, ...)` , ce que montre `strace` .

Derrière chaque descripteur, il y a trois niveaux:

1. **Table des descripteurs**, une par processus: le numéro (0, 1, 2, 3...) est un indice dans cette table.
2. **Table des fichiers ouverts**, une pour tout le système: chaque `open` crée une entrée avec le mode, la **position** courante et un compteur de références.
3. **Table des inodes en mémoire** : une copie de l'inode par fichier, partagée par toutes les ouvertures.

C'est pourquoi il importe de savoir qui partage chaque entrée. Deux appels `open` sur le même fichier créent deux entrées aux positions indépendantes; `dup` , `dup2` et `fork` copient le *pointeur* et partagent la position. Avec `donnees.txt` = `abcdefgh` :

Sortie réelle

```
$ ./alias_open
abcdabcd
```

Sortie réelle

```
$ ./alias_dup
abcdefgh
```

Conséquences pratiques: `./prog > out 2> out` ouvre `out` deux fois et la sortie d'erreur écrase la sortie standard; `./prog > out 2>&1` utilise `dup2` et ne perd rien. Avec `O_APPEND` , chaque `write` se place à la fin de façon atomique: c'est ainsi que les serveurs écrivent leurs journaux. `close` retire une référence; l'entrée est libérée quand elle atteint 0.

Pratiquez → A3.2 Les trois tables · TD2 · 3

§5 Fichiers spéciaux: FIFO, périphériques, /proc

«Tout est fichier» signifie que beaucoup de choses qui ne sont pas des données sur un disque utilisent la même interface `open / read / write / close`. Une **FIFO** (tube nommé, `mkfifo`) est un inode sans blocs: ce qu'un processus écrit, un autre le lit à travers un tampon de 64 Kio dans le noyau. Ses règles, qu'utilise le chat du TD2:

- `open(O_RDONLY)` attend que quelqu'un ouvre en écriture, et inversement (avec `O_NONBLOCK`, un écrivain sans lecteur obtient `ENXIO`).
- Une écriture d'au plus `PIPE_BUF` = 4096 octets est **atomique**: elle ne se mélange pas avec celle d'un autre écrivain. Une plus grande peut se mélanger.
- `read` renvoie 0 (fin de fichier) quand le **dernier** écrivain ferme.
- Écrire sans aucun lecteur produit le signal `SIGPIPE` (ou l'erreur `EPIPE` s'il est ignoré).

Autres fichiers spéciaux: les **périphériques** caractère et bloc de `/dev` (créés avec `mknod`), les **sockets** du domaine UNIX (unité 8 et TD8) et les systèmes de fichiers virtuels `/proc` et `/sys`, dont le noyau génère les «fichiers» au moment où on les lit.

Sortie réelle

```
$ ./fifo_server chat.fifo & sleep 0.3; echo 'hola desde el cliente' | ./fifo_client
chat.fifo; wait
server: waiting for clients on chat.fifo
hola desde el cliente
```

Pratiquez → A3.3 FIFO: le chat · TD2 · 2

§6 Parcourir les répertoires

`opendir` ouvre un répertoire, `readdir` renvoie une `struct dirent` par entrée (ou `NULL` à la fin) et `closedir` le ferme. Les entrées arrivent dans l'ordre interne du système de fichiers — sur ext4, l'ordre d'un index haché — et incluent `.` et `..`. Dans `struct dirent`, ce qui compte, ce sont `d_name`, `d_ino` et `d_type`; ce dernier évite un `stat`, mais il peut valoir `DT_UNKNOWN` sur certains systèmes de fichiers, et il faut alors `lstat`. En parcourant un arbre, on ne suit jamais les liens symboliques vers des répertoires: ils pourraient former des cycles.

C tds/td2/myls.c

```
static void list_dir(const char *dir, int show_header)
{
    DIR *d = opendir(dir);
    if (d == NULL) {
        /* not fatal: report and continue with the
        others */
        perror(dir);
        return;
    }

    /* Collect the names so that we can sort them. */
    size_t n = 0, cap = 16;
    char **names = malloc(cap * sizeof *names);
    exit_if(names == NULL, "malloc");
    struct dirent *e;
    while ((e = readdir(d)) != NULL) {
        if (n == cap) {
            cap *= 2;
            names = realloc(names, cap * sizeof *names);
            exit_if(names == NULL, "realloc");
        }
        names[n] = strdup(e->d_name);
        exit_if(names[n] == NULL, "strdup");
        n++;
    }
    exit_if(closedir(d) == -1, "closedir");
}
```

Sortie réelle

```
$ ./myls -lR demo
demo:
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 ..
-rw-rw-r--  1 student student    3 2026-09-30 13:12 a.txt
lrwxrwxrwx  1 student student    5 2026-09-30 13:12 link → a.txt
prw-rw-r--  1 student student    0 2026-09-30 13:12 pipe
drwxrwxr-x  2 student student  4096 2026-09-30 13:12 sub

demo/sub:
drwxrwxr-x  2 student student  4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 ..
```

§7 Au-delà: journalisation, copie sur écriture et `fsync`

- **Journalisation.** Créer un fichier modifie plusieurs blocs (bitmap, inode, répertoire). Si le courant coupe à mi-chemin, le système de fichiers reste incohérent. ext4 et XFS écrivent d'abord l'opération dans un journal puis l'appliquent: au redémarrage, il suffit de rejouer ou d'écarter ce qui était en attente.
- **Copie sur écriture.** Btrfs et ZFS n'écrasent jamais un bloc sur place: ils en écrivent un nouveau et changent le pointeur. On obtient des instantanés presque gratuits et des sommes de contrôle de chaque bloc.
- **Cache de pages et `fsync`.** `write` revient quand les données sont dans le cache du noyau, pas sur le disque. Un programme qui a besoin de durabilité (une base de données, un éditeur qui enregistre) appelle `fsync`, et pour que le *nom* soit durable aussi, `fsync` sur le répertoire.
- **FUSE.** Un système de fichiers peut être implémenté comme un processus utilisateur (sshfs, rclone): le noyau lui transmet les appels.